Skeleton-Based Action Recognition Using HRNet 2D Keypoints

CS487.687 - Fundamentals of Deep Learning
Instructor: Professor Alireza Tavakkoli
M.S Student: Ngoc Huan Le

Abstract

This project studies skeleton-based human action recognition using HRNet 2D keypoints on an NTU RGB+D 60-style dataset. Four neural architectures are implemented and compared: a flattened MLP baseline, an LSTM baseline, a two-stream CNN, and a spatial-temporal graph convolutional network (ST-GCN). Each video is represented as a sequence of COCO-17 body joints, resampled to 100 frames, with up to two persons per sample. The updated evaluation uses exported artifacts from `skeleton_project_outputs`, which contain the outputs of the trained models. On the cross-subject validation split with 16,487 samples, ST-GCN achieves the best validation accuracy of 82.22%, outperforming the two-stream CNN at 75.10%, LSTM at 70.66%, and MLP at 49.51%. The results suggest that models perform better when they can use both the movement over time and the body-joint structure.

1. Introduction

Human action recognition aims to classify activities from video and has applications in surveillance, human-computer interaction, healthcare monitoring, sports analytics, and behavior analysis. Direct recognition from RGB video can be sensitive to background, lighting, clothing, camera viewpoint, and irrelevant texture. Skeleton-based recognition instead represents the actor as body joints evolving over time, reducing nuisance visual variation while preserving motion patterns that are central to many actions.

This project focuses on action recognition from 2D HRNet keypoints rather than raw RGB, depth, or 3D skeletons. Each sample contains a temporal sequence of skeletons and one of 60 action labels. The goal is to compare models that use the skeleton information in different ways: an MLP that flattens the sequence, an LSTM that models temporal order, a two-stream CNN that uses pose and motion maps, and ST-GCN that explicitly models joints as a spatial-temporal graph.

The main contributions are: (1) a consistent preprocessing pipeline that converts variable-length HRNet keypoint sequences into fixed-size tensors; (2) a controlled comparison of four deep learning models under the same xsub split; and (3) an error analysis using training curves, normalized confusion matrices, and top confused class pairs. The main finding is that ST-GCN works best because its design fits the way human joints are connected.

2. Literature Review

The NTU RGB+D dataset introduced a large benchmark for human action recognition with RGB, depth, infrared, and 3D skeleton modalities (Shahroudy et al., 2016). It remains a common reference point for cross-subject and cross-view evaluation. This project uses an NTU RGB+D 60-style split, but replaces 3D skeletons with 2D HRNet keypoints.

Pose estimation is the first step in this pipeline. HRNet maintains high-resolution representations throughout the network and repeatedly exchanges information across resolutions, producing strong 2D keypoint estimates (Sun et al., 2019). Using HRNet keypoints makes the action-recognition model more compact than full video models, but it also removes appearance cues such as objects, fine hand state, and depth.

Deep learning approaches for skeleton recognition differ in how they encode temporal and spatial structure. Recurrent models such as LSTMs are natural temporal baselines because they process ordered frame sequences. CNN-based skeleton methods represent joint trajectories as pseudo-images or time-joint maps; the two-stream CNN approach uses both pose and motion information to improve recognition (Li et al., 2017). Graph-based models more directly match the skeleton topology. ST-GCN defines joints as graph nodes and bones as edges, then applies spatial graph convolution and temporal convolution to learn action features (Yan et al., 2018). Later skeleton-recognition toolkits and benchmarks, including PYSKL, emphasize careful preprocessing, augmentation, and graph-model design choices for strong performance (Duan et al., 2022). This project positions the implemented models along that progression from flattened features to sequence models, convolutional pose-motion maps, and explicit spatial-temporal graphs.

3. Technical Details

3.1. Skeleton Representation and Graph Modeling

Skeleton-based action recognition assumes that many actions can be described by the motion of body joints rather than by raw pixels. A skeleton sequence can be written as a tensor

$$X \in \mathbb{R}^{C \times T \times V \times M}, \tag{1}$$

where C is the feature-channel dimension, T is the number of frames, V is the number of joints, and M is the maximum number of people. In this project, the ST-GCN input uses $C = 3$ for x, y, and keypoint confidence score; $T = 100$ resampled frames; $V = 17$ COCO/HRNet joints; and $M = 2$ persons. This representation is compact and reduces variation from scene texture, clothing, and background, but it also discards appearance cues such as objects, depth, and detailed hand shape.

The graph view is useful because body joints are connected like a skeleton, not arranged like pixels in an image. In a skeleton graph $G = (V, E)$, each joint is a node and each anatomical connection is an edge. A graph convolution updates a joint feature by aggregating information from neighboring joints:

$$f_{\text{out}}(v_i) = \sum_{v_j \in \mathcal{B}(v_i)} \frac{1}{Z_i(v_j)} f_{\text{in}}(v_j) W(l_i(v_j)), \tag{2}$$

where $\mathcal{B}(v_i)$ is the set of neighboring joints around node v_i, $Z_i(v_j)$ is a normalization term, and $l_i(v_j)$ assigns the neighbor to a spatial partition. ST-GCN uses partitions such as self-links, inward edges, and outward edges so that different types of joint relationships can learn different weights.

Temporal modeling is added by applying convolution over consecutive frames after spatial graph aggregation. This lets the model learn both pose structure within each frame and motion patterns across time. The learnable edge-importance weights further allow the network to adjust how strongly each bone connection contributes during classification. This explains why the models are expected to improve in this order. MLP uses one long vector, so it misses much of the sequence and body structure. LSTM uses the time order better. CNN uses pose and motion maps. ST-GCN is the most suitable because it directly uses the skeleton graph over time.

3.2. Dataset and Preprocessing

The dataset used in this project is derived from NTU RGB+D 60, but the experiments use HRNet 2D skeletons rather than raw RGB or depth frames. Each sample corresponds to one action clip and is represented by a sequence of detected human body joints over time. The action label belongs to one of 60 NTU action categories, including daily actions, hand-object actions, body-motion actions, and person-interaction actions.

The dataset contains 56,578 annotations with predefined xsub and xview splits. Experiments in this report use the xsub split: 40,091 samples for training and 16,487 for validation. The dataset is stored in a pickle file with two main fields: `annotations`, which contains the samples, and `split`, which contains predefined splits including `xsub_train`, `xsub_val`, `xview_train`, and `xview_val`. The dataset source is the HRNet NTURGB+D 2D skeleton release.

Each annotation stores a sequence identifier, action label, original frame count, HRNet 2D keypoints, and keypoint confidence scores. The keypoint tensor is approximately $(M, T, 17, 2)$, where M is the number of persons, T is the original frame length, 17 is the COCO joint count, and 2 denotes (x, y) coordinates. The `keypoint_score` field stores the HRNet confidence score for each detected joint. These scores are useful because pose estimation may be uncertain when joints are occluded, blurred, or outside the image.

Because this project uses 2D skeletons only, the input does not include RGB appearance, depth, object identity, or detailed finger motion. This makes the representation efficient and robust to background variation, but it also explains why actions such as reading, writing, using a phone, and typing can be difficult to distinguish from skeleton motion alone.

Another simple but important step is temporal resampling. The original clips have different numbers of frames, so the models need a fixed length for batching. Resampling every sequence to 100 frames keeps the main motion pattern while making the input size consistent. It also makes the comparison fair because all four models use the same temporal length.

All models use a shared fixed-length representation. Each sequence is linearly resampled to $T = 100$ frames. Up to $M = 2$ persons are kept; if only one person is present, the second person slot is padded with zeros. The skeleton convention used in the graph-based model is $C \times T \times V \times M$, where C is the channel count, $V = 17$ joints, and $M = 2$ persons. The core preprocessing steps are: (1) person handling, where CNN, LSTM, and ST-GCN can keep the top `NUM_PERSON`=2 persons by `keypoint_score` when needed, while the MLP baseline uses the first two person slots; (2) temporal resizing to 100 frames using linear interpolation; (3) missing-person padding with zeros; and (4) model-specific tensor construction. The current dataset contains at most two persons per sample, so the person-ranking difference does not affect the reported split. The models differ in how this common skeleton information is reshaped and normalized, as summarized in Table 1.

Table 1. Input representations and training settings.

Model	Input shape	Score use	Normalization
MLP	(6800)	coord. weight	sample mean/std
LSTM	$(100, 68)$	not used	train mean/std
CNN	(M, C, T, V)	person rank	none
ST-GCN	$(3, 100, 17, 2)$	channel	train mean/std

Setting	MLP	LSTM	CNN	ST-GCN
Batch size	64	64	64	64
Epochs	20	20	20	20
Optimizer	Adam	Adam	Adam	Adam
LR	10^{-3}	10^{-3}	10^{-3}	10^{-3}
Weight decay	10^{-4}	10^{-4}	10^{-4}	10^{-4}
Scheduler	none	none	none	StepLR(10, 0.5)

3.3. Models

MLP baseline. The MLP flattens the full skeleton sequence into a single vector. With two persons, two coordinate channels, 100 frames, and 17 joints, each sample has $2 \times 2 \times 100 \times 17 = 6800$ input features. The classifier is Linear $6800 \to 512$, BatchNorm, ReLU, Dropout 0.3; Linear $512 \to 256$, BatchNorm, ReLU, Dropout 0.3; and Linear $256 \to 60$. This model is fast but discards explicit temporal order and body connectivity.

LSTM baseline. The LSTM treats each sample as a sequence of 100 frame vectors. Each frame contains $2 \times 2 \times 17 = 68$ features. The model uses input size 68, hidden size 256, two recurrent layers, dropout 0.5, and a final Linear $256 \to 60$ classifier. It preserves temporal order, but anatomical relationships are only learned implicitly from flattened frame features.

Two-stream CNN. The CNN represents skeleton sequences as time-joint maps. The raw stream contains resized 2D coordinates, while the motion stream contains frame differences. Each stream passes through a learnable skeleton transformer that maps 17 joints to 25 transformed joints before convolution. The CNN backbone applies 2D convolution over time and joint dimensions, concatenates the two streams, and uses maxout-style merging over persons before classification.

ST-GCN. ST-GCN represents the skeleton as a graph with joints as nodes and anatomical bones as edges. The COCO-17/HRNet graph uses an adjacency tensor of shape $(3, 17, 17)$ for self-links, inward edges, and outward edges. The model input is $(C, T, V, M) = (3, 100, 17, 2)$, where channels are x, y, and keypoint score. The network contains nine ST-GCN blocks with channels 64, 64, 64, 128, 128, 128, 256, 256, and 256. Each block uses spatial graph convolution, temporal convolution with kernel size 9, BatchNorm, Dropout 0.5, residual connections, and learnable edge-importance weights. Features are pooled over time, joints, and persons before the final classifier.

3.4. Experimental Setup

All experiments are implemented in PyTorch and run in a GPU/Colab environment. All models are trained with cross-entropy loss,

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(y_i \mid x_i), \tag{3}$$

and Adam with learning rate 10^{-3} and weight decay 10^{-4}. The best checkpoint for each model is selected by validation accuracy. ST-GCN additionally uses a StepLR scheduler with step size 10 and decay factor 0.5, while the other baselines use the fixed learning-rate setting summarized in Table 1.

3.5. Evaluation Metrics

The primary metric is validation accuracy,

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{number of validation samples}}. \tag{4}$$

Macro F1 and weighted F1 are also reported to measure per-class and support-weighted classification quality. The updated output folder also includes precision, recall, F1-score, support, confusion matrices, and top confused class pairs for each model. This report uses the exported text, CSV, and image files from the `skeleton_project_outputs` folders for the MLP, LSTM, CNN, and ST-GCN runs.

4. Results and Discussion

4.1. Quantitative Results

Table 2. Validation performance on the xsub split.

Model	Accuracy	Macro F1	Weighted F1
MLP	49.51%	0.4856	0.4857
LSTM	70.66%	0.7030	0.7031
Two-stream CNN	75.10%	0.7425	0.7427
ST-GCN	**82.22%**	**0.8216**	**0.8217**

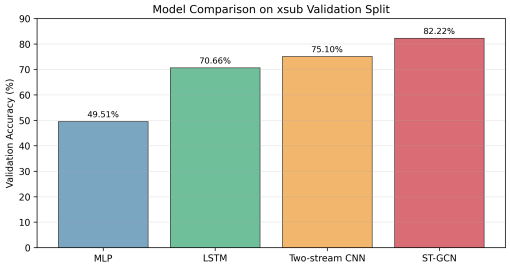

Figure 1. Model comparison by validation accuracy. Performance improves as the architecture preserves more skeleton structure.

Table 2 and Figure 1 show the performance ranking MLP $<$ LSTM $<$ two-stream CNN $<$ ST-GCN. The bar chart has a monotonic increasing shape from MLP to ST-GCN, which means performance improves as the model uses more of the skeleton structure. The largest jump is from MLP to LSTM, from 49.51% to 70.66%, showing the value of temporal modeling. CNN adds another 4.44 points over LSTM by using raw-pose and motion streams. ST-GCN adds 7.12 points over CNN and 32.71 points over MLP, so the graph-based spatial-temporal design gives the strongest overall result.

The results also show that how the skeleton is represented matters a lot. The MLP can still learn some useful patterns, but using one flattened vector makes the task harder. The LSTM improves the result because it keeps the order of frames. The CNN improves further by using both pose and motion. ST-GCN gives the best result because it uses the joint connections and the motion over time together.

4.2. Training Behavior

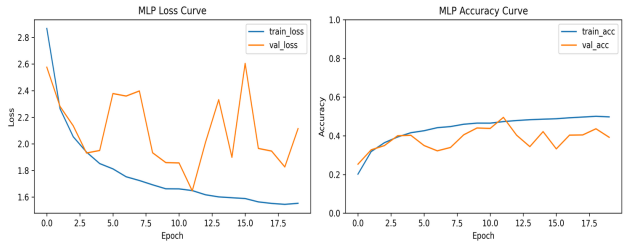

Figure 2. Training and validation curves for the MLP baseline.

Figure 2 shows that the MLP curves have an unstable, noisy validation shape. Training accuracy increases from 20% at epoch 1 to 49% at epoch 20, but validation accuracy peaks earlier at 49.5% in epoch 12 and then falls to 39% by epoch 20. Validation loss also reaches its best value at epoch 12 (1.6469) and then rises to 2.1143. This shows that MLP learns some coarse pose patterns, but after the middle epochs it generalizes poorly because the flattened vector does not explicitly model temporal sequence or joint connectivity.

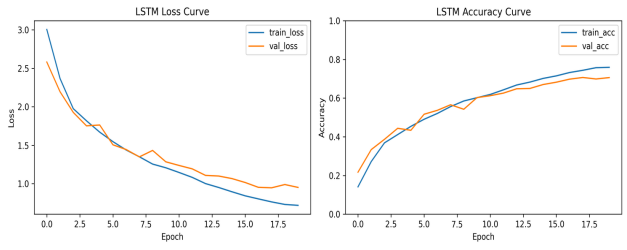

Figure 3. Training and validation curves for the LSTM baseline.

Figure 3 shows that the LSTM curves are smoother and mostly upward-sloping for accuracy. Validation accuracy rises from 21.70% at epoch 1 to its best value of 70.65% at epoch 18, then ends at 70.59% at epoch 20. Validation loss drops from 2.5812 to its best value of 0.9453 at epoch 18 and ends at 0.9503. The final train-validation accuracy gap is about 5.30 percentage points (75.89% train vs. 70.59% validation), so the model shows moderate overfitting. The curve shape supports the theory that sequence modeling helps, although the flat per-frame input still limits spatial skeleton reasoning.

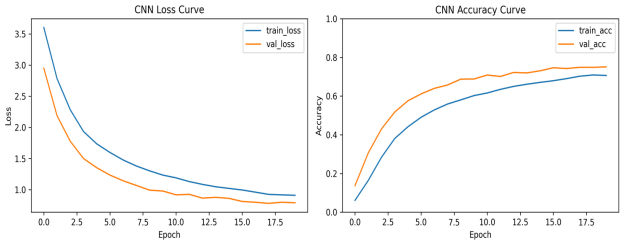

Figure 4. Training and validation curves for the two-stream CNN.

Figure 4 shows a steep early improvement followed by a slower late increase. Validation accuracy grows from 13.68% at epoch 1 to 75.10% at epoch 20, and validation loss decreases from 2.9503 to 0.7908. The best checkpoint is also epoch 20, so the validation curve is still useful at the end rather than collapsing. Validation accuracy is higher than training accuracy at the last epoch (75.10% vs. 70.66%), which is consistent with regularized training behavior such as dropout. The shape suggests that the motion stream improves recognition beyond LSTM.

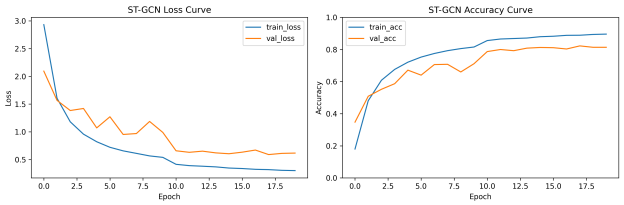

Figure 5. Training and validation curves for ST-GCN.

Figure 5 shows that ST-GCN has the strongest curve shape: a fast rise in the first half, then a high plateau with small late fluctuations. Validation accuracy increases from 34.72% at epoch 1 to the best value of 82.22% at epoch 18, then ends at 81.35% at epoch 20. Validation loss is also lowest at epoch 18 (0.5901) and rises to 0.6169 by epoch 20 while training accuracy reaches 89.56%. This indicates mild late overfitting, but the model still generalizes best because its graph convolution matches the spatial-temporal structure of skeleton data.

4.3. Confusion Matrix Analysis

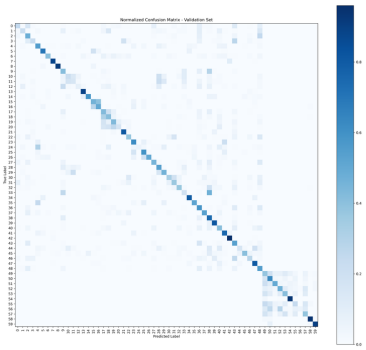

Figure 6. Normalized confusion matrix for the MLP baseline.

Figure 6 shows that the MLP confusion matrix has the weakest diagonal among the four models and many scattered off-diagonal errors. This means the flattened representation often groups actions by rough body pose instead of by temporal motion or joint relationships. The largest errors are between visually similar upper-body or paired actions, including rub two hands → put palms together (47.10%), put on a shoe → take off a shoe (39.93%), take off glasses → put on glasses (31.39%), and reach into pocket → drop (30.66%). These mistakes are consistent with the limitation of MLP: it sees one long feature vector and does not explicitly model temporal sequence or skeleton graph structure.

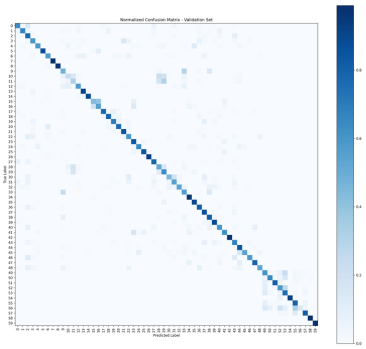

Figure 7. Normalized confusion matrix for the LSTM baseline.

Figure 7 shows that the LSTM matrix has a clearer diagonal than MLP, indicating that temporal modeling improves recognition. However, the remaining off-diagonal cells concentrate around actions that share similar hand trajectories or require object context. The strongest confusions are put on a shoe → take off a shoe (39.93%), touch pocket → giving object (32.73%), writing → type on a keyboard (31.62%), clapping → rub two hands (30.04%), and pat on back → point finger (26.09%). This indicates that LSTM learns sequence dynamics, but its per-frame flattened input still does not explicitly model anatomical joint connectivity.

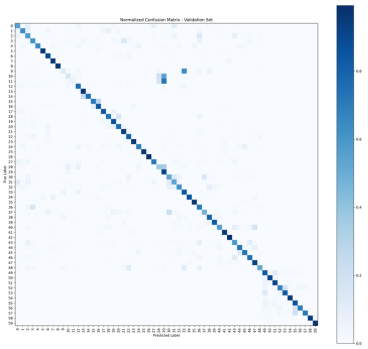

Figure 8. Normalized confusion matrix for the two-stream CNN.

Figure 8 shows that the CNN produces a stronger diagonal than MLP and LSTM, but several off-diagonal blocks remain visible. The motion stream helps many actions, yet the model still confuses actions with very similar local hand motion: writing → type on a keyboard (72.79%), clapping → rub two hands (64.47%), reading → type on a keyboard (54.95%), play with phone/tablet → type on a keyboard (35.27%), and put on a shoe → take off a shoe (28.21%). These errors suggest that raw pose plus motion is useful, but 2D skeletons do not provide object appearance or fine finger/hand-state information.

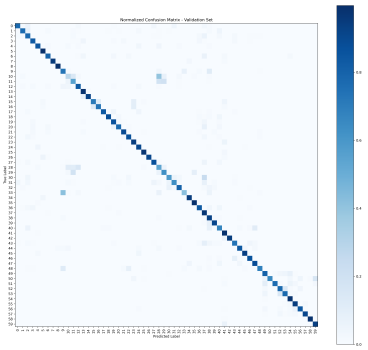

Figure 9. Normalized confusion matrix for ST-GCN.

Figure 9 shows that the ST-GCN confusion matrix has the strongest and cleanest diagonal, matching its highest validation accuracy. The graph structure helps reduce diffuse class errors because information is aggregated through anatomically connected joints over time. The remaining errors are mostly concentrated in naturally ambiguous classes: rub two hands → clapping (42.03%), reading → play with phone/tablet (39.93%), point to something → salute (23.55%), writing → type on a keyboard (19.49%), and type on a keyboard → writing (18.55%). Thus, ST-GCN handles skeleton structure best, but skeleton-only 2D input still struggles when the class depends on objects, hand details, or subtle action intent.

The remaining errors reveal an important limitation of 2D

skeleton-only input. Frequent confusions include reading, writing, playing with a phone or tablet, and typing on a keyboard; clapping, rubbing two hands, and putting palms together; and pointing to something versus taking a selfie. These actions often differ by objects, hand pose, depth, or intent rather than by large body-joint motion. A 17-joint 2D skeleton is efficient and robust, but it cannot fully encode those visual cues.

4.4. Top Confused Class Pairs

The top confusion CSV files show that many errors occur between visually similar hand-object or upper-body actions. These pairs are difficult because 2D skeletons preserve body geometry but do not contain object appearance, hand state, or depth.

Table 3. Top confused class pairs by within-class error rate.

Model	True → predicted	Count	Rate
MLP	rub hands → palms together	130	47.10%
MLP	put shoe → take off shoe	109	39.93%
MLP	take off glasses → put glasses	86	31.39%
LSTM	put shoe → take off shoe	109	39.93%
LSTM	touch pocket → giving object	90	32.73%
LSTM	writing → typing	86	31.62%
CNN	writing → typing	198	72.79%
CNN	clapping → rub hands	176	64.47%
CNN	reading → typing	150	54.95%
ST-GCN	rub hands → clapping	116	42.03%
ST-GCN	reading → phone/tablet	109	39.93%
ST-GCN	point something → salute	65	23.55%

Additional recurring confusion patterns include put on a shoe versus take off a shoe; reading, writing, playing with a phone or tablet, and typing on a keyboard; clapping, rubbing two hands, and putting palms together; and pointing to something versus taking a selfie. These errors are consistent with the input modality. Many of these actions differ by small object interactions or hand details that are not fully represented by 17 body joints.

4.5. Discussion

The results show that the MLP is the weakest baseline because it flattens the skeleton sequence into one vector. Dense layers can still learn some useful patterns, but the model does not directly learn how joints connect or how the action changes over time.

The LSTM improves performance by preserving temporal order. This helps actions where motion direction and sequence progression matter. However, the LSTM still receives each frame as a flattened vector, so anatomical relationships between joints are only learned implicitly.

The two-stream CNN improves over the LSTM by using both raw pose and motion. The motion stream is especially useful for actions that require frame-to-frame changes rather than static pose alone. The limitation is that the CNN treats the joint axis like a regular grid, while the human skeleton is an irregular graph.

ST-GCN performs best because it is designed to learn from connected joints and their movement over time. Spatial graph convolution aggregates features from anatomically connected joints, temporal convolution models motion, and the keypoint-score channel provides information about pose-estimation reliability.

4.6. Limitations

This project has several limitations. Only the xsub split is reported, so cross-view generalization is not evaluated. Skeleton data augmentation is limited or not used consistently across all models. The input uses 2D HRNet/COCO-17 keypoints, so depth and fine hand-object information are missing. Several difficult classes require object context that skeleton-only features cannot fully represent. The CNN is adapted to 2D keypoints, so it does not fully reproduce the original 3D skeleton paper setup. Finally, the ST-GCN graph assumes the HRNet/COCO-17 joint order, so a different keypoint layout would require graph changes.

5. Future Work

Future work should evaluate all models on the xview split, add skeleton data augmentation such as temporal crop, random shift, scaling, and controlled horizontal flipping, and test adaptive graph convolution or attention-based graph models. For difficult object-centric classes, a multimodal model that combines skeleton features with RGB or depth information would likely reduce the remaining confusions. Higher-resolution pose information for hand-heavy actions may also help, and an ensemble that combines ST-GCN with motion-based CNN features could further improve robustness.

6. Conclusions

This project compares four deep learning models for skeleton-based action recognition using HRNet 2D keypoints. The updated exported results show that the MLP baseline reaches 49.51% validation accuracy, LSTM reaches 70.66%, the two-stream CNN reaches 75.10%, and ST-GCN reaches 82.22%. The main lesson is that the model works better when it can use both body-joint connections and motion over time. ST-GCN performs best because it explicitly models joint connectivity, temporal dynamics, confidence-score input, and learnable edge-importance weights.

Impact Statement

This project advances skeleton-based action recognition methods for compact video understanding. Potential appli-

cations include assistive monitoring and human-computer interaction, but deployment should consider privacy, consent, dataset bias, and the risk of misclassification in safety-critical settings.

Dataset

Dataset: HRNet NTURGB+D [2D Skeleton].

References

Duan, H., Wang, J., Chen, K., and Lin, D. PYSKL: Towards good practices for skeleton-based action recognition. In *Proceedings of the 30th ACM International Conference on Multimedia*, 2022.

Li, C., Zhong, Q., Xie, D., and Pu, S. Skeleton-based action recognition with convolutional neural networks. In *2017 IEEE International Conference on Multimedia & Expo Workshops*, 2017.

Shahroudy, A., Liu, J., Ng, T.-T., and Wang, G. NTU RGB+D: A large scale dataset for 3d human activity analysis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016.

Sun, K., Xiao, B., Liu, D., and Wang, J. Deep high-resolution representation learning for human pose estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

Yan, S., Xiong, Y., and Lin, D. Spatial temporal graph convolutional networks for skeleton-based action recognition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.